

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

Sistemas Operacionais - 2026.2
AV2 — Chat local com Ollama + Open WebUI

Enzo Emanuel Costa Maia
Filipe de Carvalho Godoy
Giulian Fábio Bastos Amorim Lima
José Fagner Silva Junqueira
Katyane dos Santos
Rauany Ingrid Santos de Jesus
Vênisson Cardoso dos Santos

São Cristóvão, Sergipe
2026

Nota: Todos os códigos-fonte, prompts e demais artefatos de análise utilizados para a elaboração deste tutorial estão disponíveis publicamente em nosso repositório no GitHub: > https://github.com/Enz0EMCosta/SO_2026_2_TrilhaA

Nota: Link para a pasta com vídeo do grupo: https://drive.google.com/file/d/1qio1HxR_OiQHEN639TeSWmgItdoIr36o/view?usp=drive_link

1 - Ambiente - Enzo Emanuel Maia Costa

Tarefa: Documentar o ambiente experimental da AV2, registrando sistema operacional, kernel, processador, memória, swap, armazenamento e versões das ferramentas, e explicar a diferença entre MemFree, MemAvailable, cache de páginas e swap, relacionando-os ao GenResearch-Servidor.

Análise da tarefa: O ambiente utilizado foi o WSL2 com Ubuntu 24.04.1 LTS e kernel 6.18.40.1-microsoft-standard-WSL2. O repositório foi clonado no sistema de arquivos do Linux (~) e não em /mnt/c, pois o acesso ao disco do Windows passa por uma camada de tradução que distorce as medições de leitura e escrita. Para o inventário foram utilizados **uname -a e cat /etc/os-release** (sistema e kernel), **lscpu e nproc** (processador), **free -h e cat /proc/meminfo** (memória), l**sblk, df -h e swapon --show** (armazenamento e swap), além de **vmstat 1 5 e iostat -xz 1 5** para a linha de base em repouso. O iostat faz parte do pacote sysstat, instalado com o apt. Também foram registradas as versões do Python (3.12.3) e do Git (2.43.0), o commit do repositório (62f1b2e, de 06/10/2026) e a data da coleta (15/10/2026).

Evidências:

```
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 39 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          8
  On-line CPU(s) list:           0-7
Vendor ID:                       GenuineIntel
  Model name:                    Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
    CPU family:                  6
    Model:                       142
    Thread(s) per core:          2
    Core(s) per socket:          4
    Socket(s):                   1
    Stepping:                    12
    BogoMIPS:                    3600.01
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mt
rch_perfmon rep_good nopl xtopology cpuid tsc_kn
     6c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ib
                                 bmi1 avx2 smep bmi2 erms invpcid rdseed adx sm
                                 md_clear flush_l1d arch_capabilities
Virtualization features:
  Hypervisor vendor:             Microsoft
  Virtualization type:           full
Caches (sum of all):
  L1d:                           128 KiB (4 instances)
  L1i:                           128 KiB (4 instances)
  L2:                            1 MiB (4 instances)
  L3:                            6 MiB (1 instance)
NUMA:
  NUMA node(s):                  1
  NUMA node0 CPU(s):             0-7
Vulnerabilities:
  Gather data sampling:          Unknown: Dependent on hypervisor status
  Ghostwrite:                    Not affected
  Indirect target selection:     Mitigation; Aligned branch/return thunks
  Itlb multihit:                 KVM: Mitigation: VMX unsupported
```

```
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       458Mi       2.8Gi       4.0Mi       642Mi       3.3Gi
Swap:          1.0Gi          0B       1.0Gi
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ nproc
8
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 722.9M  1 disk
sdb   8:16   0  96.5M  1 disk
sdc   8:32   0     1G  0 disk [SWAP]
sdd   8:48   0   256G  0 disk /mnt/wslg/distro
                              /
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G     0  1.9G   0% /usr/lib/modules/6.18.40.1-microsoft-standard-WSL2
none            1.9G  4.0K  1.9G   1% /mnt/wsl
drivers         222G  187G   35G  85% /usr/lib/wsl/drivers
/dev/sdd        251G  7.8G  231G   4% /
none            1.9G   36K  1.9G   1% /mnt/wslg
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  3.3M  1.9G   1% /init
none            1.9G  488K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G     0  1.9G   0% /run/shm
none            1.9G   80K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   80K  1.9G   1% /mnt/wslg/doc
C:\             222G  187G   35G  85% /mnt/c
tmpfs           385M   20K  385M   1% /run/user/1000
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc partition   1G   0B   -2
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ head -n 20 /proc/meminfo
MemTotal:        3936628 kB
MemFree:         3507172 kB
MemAvailable:    3491340 kB
Buffers:            2076 kB
Cached:            94188 kB
SwapCached:            0 kB
Active:            33792 kB
Inactive:         115144 kB
Active(anon):       2956 kB
Inactive(anon):    53888 kB
```

Quanto aos conceitos, **MemFree** é a memória totalmente sem uso, enquanto **MemAvailable** estima quanto pode ser entregue a novos processos sem recorrer ao swap, incluindo a parte do cache que o kernel consegue liberar. O cache de páginas guarda em RAM cópias de arquivos lidos do disco, de modo que uma nova leitura é atendida sem acessar o disco. Por isso ele não é memória desperdiçada: é devolvido automaticamente quando outro processo precisa. O **swap**, por sua vez, é espaço em disco usado quando a RAM aperta, muito mais lento, e sua atividade (colunas si e so do vmstat) indica pressão de memória. Essa relação é central para o servidor, porque o corpus, o índice e os **embeddings** são arquivos lidos repetidamente: a primeira leitura vem do disco e as seguintes tendem a vir do cache, o que justifica comparar primeira e segunda execução nos **experimentos de leitura convencional e mmap.**

Resultados e Conclusão

A partir do inventário e da documentação do ambiente, foi possível estabelecer as condições de execução sobre as quais o GenResearch-Servidor será desenvolvido e medido. O ambiente utilizado foi o WSL2 com Ubuntu 24.04.1 LTS e kernel 6.18.40.1-microsoft-standard-WSL2, em um processador Intel Core i5-8265U com 4 núcleos e 8 threads, 3,8 GiB de RAM e 1 GiB de swap visíveis ao sistema, além de 231 GB livres no disco do Linux. O repositório foi mantido no sistema de arquivos do Linux e não em /mnt/c, de modo que as leituras e escritas dos experimentos não passem pela camada de tradução do disco do Windows, o que distorceria as medições de entrada e saída. Em repouso, a máquina apresentou CPU cerca de 99,6% ociosa, nenhuma atividade de swap, iowait próximo de zero e aproximadamente 3,4 GB de memória livre, valores que servem de 1linha de base para comparar os experimentos de leitura convencional e mmap, de concorrência, de cache e de pressão de memória. A análise de MemFree, MemAvailable e cache de páginas mostrou que a memória aparentemente ocupada por cache não representa desperdício, pois é devolvida ao sistema quando necessária, e que o valor de cache variou entre duas medições seguidas, o que reforça a necessidade de comparar a primeira e a segunda execução de cada experimento separadamente.

Também foi possível identificar limitações importantes. A memória visível ao WSL2, de 3,8 GiB, é menor que os 6,1 GiB registrados no mesmo tipo de ambiente na AV1, e o swap de apenas 1 GiB restringe o quanto o corpus, o contexto e o lote podem crescer no experimento de pressão de memória sem que o processo seja encerrado pelo kernel. Além disso, toda a execução ocorre em CPU, sem GPU dedicada, e sobre um hipervisor, de modo que os resultados refletem um ambiente virtualizado e não um Linux em hardware nativo, e o disco C: do Windows, com 85% de uso, recomenda evitar a geração de arquivos grandes. Logo, concluímos que a tarefa permitiu documentar de forma reprodutível o ambiente da AV2, abrangendo desde a identificação dos recursos computacionais até a explicação dos conceitos de memória e cache de páginas que fundamentam os experimentos, e que o inventário realizado também delimitou as restrições de hardware que devem ser consideradas ao dimensionar o corpus e interpretar os resultados. Dessa forma, a preparação do ambiente fornece a base necessária para as análises seguintes da equipe, especialmente aquelas relacionadas à memória, paginação, armazenamento, cache e concorrência.

2 - Ingestão + Armazenamento - José Fagner Silva Junqueira

Ingestão e armazenamento

Implementação da ingestão

O módulo de ingestão realiza a leitura de documentos `.txt` codificados em UTF-8, incluindo arquivos presentes em subpastas. Para cada documento, o sistema identifica sua origem, lê o conteúdo e o divide em fragmentos menores, que posteriormente recebem vetores e são armazenados.

A fragmentação utiliza trechos de até 200 palavras, com sobreposição de 40 palavras entre trechos consecutivos. Dessa forma, o avanço é de 160 palavras, preservando parte do conteúdo nas fronteiras dos fragmentos. Cada trecho mantém sua associação ao documento de origem e sua posição na sequência.

Arquivos vazios são ignorados com registro da ocorrência. Falhas de decodificação também são registradas, permitindo que os demais documentos continuem sendo processados.

Geração dos vetores

Foram utilizados vetores simulados determinísticos, alternativa aceita pelo enunciado. A geração utiliza SHA-256 com contador para produzir vetores de 128 dimensões, normalizados pela norma L2 e armazenados no formato `float32` little-endian.

O determinismo permite que o mesmo texto produza o mesmo vetor em diferentes processos e execuções, favorecendo a reprodução dos testes. Entretanto, esses vetores não representam relações semânticas entre textos. Sua finalidade nesta implementação é permitir a avaliação funcional do pipeline e de seus custos de processamento e armazenamento.

Cada vetor ocupa 512 bytes, correspondentes a 128 componentes de quatro bytes.

Armazenamento e justificativa

O armazenamento utiliza SQLite, escolhido por oferecer persistência local em um único arquivo, suporte a transações e integração com a biblioteca padrão do Python, sem exigir um servidor de banco de dados separado.

A estrutura contém as seguintes tabelas:

- `documents`: identificação, caminho relativo, tamanho em bytes e hash do conteúdo dos documentos.
- `chunks`: texto dos fragmentos, posição e vínculo com o documento de origem.
- `vectors`: vetor associado a cada fragmento, dimensão e formato de armazenamento.
- `index_config`: configuração utilizada na ingestão.
- `ingestion_runs`: registros das execuções.

A reingestão de documentos sem alterações evita a duplicação de registros. Quando o conteúdo de um documento é modificado, seus dados e registros dependentes são substituídos. Mudanças incompatíveis na configuração, como dimensão dos vetores, tamanho dos trechos ou sobreposição, exigem reconstrução explícita por meio da opção `--rebuild`.

Os demais componentes podem acessar os trechos e vetores em lotes pela interface `SQLiteIndex.iter_chunk_vectors(batch_size=128)`.

Execução e resultados

A demonstração funcional foi realizada em terminal PowerShell no Windows, utilizando o mesmo corpus de exemplo em dois modos: leitura convencional (`standard`) e mapeamento de arquivo em memória (`mmap`). Cada modo utilizou um arquivo SQLite separado.

Os comandos executados foram:

python -m servidor.ingestao.pipeline corpus-exemplo --database data\genresearch.sqlite3

python -m servidor.ingestao.pipeline corpus-exemplo --database data\genresearch-mmap.sqlite3 --read-mode mmap

As duas execuções processaram um documento, geraram um fragmento e um vetor e terminaram sem erros.

Métrica	Leitura convencional	mmap
Documentos processados	1	1

Fragmentos gerados	1	1
Vetores gerados	1	1
Erros registrados	0	0
Tamanho do corpus	171 bytes	171 bytes
Volume dos vetores	512 bytes	512 bytes
Tamanho total do SQLite	40.960 bytes	40.960 bytes
Tempo de leitura	0,2134 ms	0,3234 ms
Tempo total de ingestão	35,7695 ms	31,7087 ms

O tamanho do corpus corresponde aos arquivos de entrada. O volume dos vetores corresponde à soma dos BLOBs armazenados. Já o tamanho total do SQLite inclui textos, metadados, vetores e estruturas internas do banco, totalizando 40 KiB nesta demonstração.

Essas medidas representam espaço ocupado pelos dados e não equivalem à quantidade de E/S física realizada pelo dispositivo.

Os tempos apresentados correspondem a uma execução por modo com um documento pequeno. Portanto, demonstram o funcionamento, sem permitir concluir superioridade de desempenho de um método. Embora o mmap tenha apresentado menor tempo total nessa execução, seu tempo isolado de leitura foi maior.

Validação funcional

A validação utilizou o comando:

python -m unittest discover -s tests -v

A suíte executou seis testes, todos aprovados, em 0,219 segundo. Os cenários abrangem:

- Fragmentação, sobreposição e validação de parâmetros.
- Tratamento de corpus inexistente.

- Reprodução e normalização dos vetores.
- Equivalência dos dados armazenados entre leitura convencional e mmap.
- Persistência, reabertura, leitura em lotes e reingestão sem duplicação.
- Subpastas, arquivos vazios, documentos modificados, configuração incompatível e UTF-8 inválido.

As mensagens de arquivo vazio e falha de decodificação ocorreram durante cenários de erro previstos pelos testes. O resultado final foi `OK`, indicando que esses comportamentos atenderam às verificações implementadas.

Síntese da contribuição

A contribuição disponibiliza um pipeline de leitura, fragmentação, vetorização simulada e armazenamento persistente. A execução registra as quantidades processadas, os tamanhos do corpus e do banco e os tempos das etapas. Os fragmentos preservam sua origem e ficam disponíveis, junto aos vetores, para integração com a recuperação.

Os resultados apresentados referem-se ao corpus de exemplo. Como os vetores são simulados, a validação comprova o funcionamento do pipeline e da persistência, sem avaliar a qualidade semântica da recuperação.

- **3 Fila de requisições, concorrência e sincronização - Vênisson Cardoso dos Santos.**

  Este componente teve como objetivo organizar o recebimento e o processamento concorrente de consultas no GenResearch-Servidor, atendendo aos requisitos da Parte B, da configuração experimental C3 e da Opção 1 da Parte D. A implementação foi desenvolvida por Foi utilizado o modelo de produtores e consumidores, no qual threads produtoras inserem consultas em um buffer limitado e threads consumidoras retiram essas consultas para processamento. Cada requisição recebe um identificador único, permitindo verificar perdas e duplicações ao final da execução.

  Implementação e mecanismos de sincronização

  A fila foi implementada em Python com a estrutura *deque*, utilizando apenas recursos da biblioteca padrão. A configuração inicial empregou dois produtores, dois consumidores e um buffer com capacidade para três itens. Cada produtor enviou seis consultas, totalizando doze requisições por execução.

  A sincronização foi realizada com os semáforos *empty* e *full* e um mutex. O semáforo *empty*, inicializado com três permissões, controla as vagas disponíveis no buffer. O semáforo *full*, inicialmente sem permissões, controla a disponibilidade de consultas para os consumidores.

O mutex protege as operações de inserção e retirada, garantindo exclusão mútua no acesso à estrutura compartilhada.

Antes de inserir uma consulta, o produtor aguarda uma permissão de *empty*. Após a inserção protegida pelo *mutex*, sinaliza a disponibilidade de um item em *full*. O consumidor executa a operação complementar: aguarda um item, retira a consulta com exclusão mútua e libera uma vaga no *buffer*.

O processamento das consultas ocorre fora da região crítica. Dessa forma, o atendimento de uma requisição não impede que outros produtores e consumidores acessem a fila. Ao término da produção, mensagens sentinelas são inseridas para encerrar os consumidores.

As threads foram escolhidas por permitirem sobrepor os períodos de espera presentes na carga simulada. Essa escolha é adequada ao experimento realizado, mas não implica execução paralela de processamento intensivo de CPU. A escrita dos registros no terminal também foi protegida por um lock.

Ambiente e metodologia experimental

Os experimentos foram executados em uma máquina virtual Ubuntu 24.04.5 LTS, no VirtualBox, com Python 3.12.3, dois processadores lógicos e aproximadamente 3,8 GiB de RAM. O acesso ao ambiente foi realizado por SSH a partir do PowerShell do Windows.

Foram avaliadas configurações com um, dois e quatro consumidores, mantendo dois produtores, doze consultas e capacidade de três itens. Cada configuração foi executada três vezes. A configuração com um consumidor foi utilizada como referência, enquanto as configurações com dois e quatro consumidores atenderam ao requisito mínimo de concorrência de C3.

Para isolar o comportamento da fila, o atendimento de cada consulta foi simulado com time.sleep(0.2). Foram medidas a espera na fila, a latência, a vazão, a ocupação do buffer e a utilização dos consumidores.

A espera na fila corresponde ao intervalo entre a inserção e a retirada de uma consulta. A latência compreende o intervalo entre a tentativa de envio pelo produtor e a conclusão do processamento, incluindo a espera por uma vaga. A vazão foi calculada pela razão entre consultas concluídas e tempo total da execução. O tamanho médio da fila foi ponderado pelo tempo, desconsiderando as mensagens sentinelas.

A utilização de cada consumidor corresponde ao tempo dentro da função de processamento dividido pela duração do experimento. Portanto, representa a ocupação do trabalhador, incluindo a espera simulada, e não utilização da CPU.

Resultados de concorrência

A Tabela 1 apresenta as médias das três repetições de cada configuração.

Tabela 1 — Resultados médios dos experimentos de concorrência

Consumidores	Tempo total (s)	Vazão (consultas/s)	Espera na fila (s)	Latência média (s)	Tamanho médio da fila
1	2,4193	4,96	0,5031	0,9052	2,4954
2	1,2154	9,87	0,2341	0,5298	2,3118
4	0,6196	19,37	0,1008	0,3352	1,9508

Fonte: elaboração própria, com base nos registros experimentais.

As nove execuções concluíram as doze consultas sem perdas ou duplicações. O buffer atingiu sua capacidade máxima de três itens, sem ultrapassá-la.

O aumento da quantidade de consumidores reduziu o tempo total e a espera das consultas, elevando a vazão. Com quatro consumidores, a fila apresentou menor ocupação média, indicando que os itens permaneceram menos tempo aguardando retirada.

As latências apresentaram os seguintes valores em cada repetição:

Tabela 2 — Mediana e p95 da latência por execução

Consumidores	Repetição	Mediana (s)	p95 (s)
1	1	1,0048	1,2110
1	2	1,0032	1,2061
1	3	1,1050	1,2080
2	1	0,6014	0,8010

2	2	0,6012	1,0009
2	3	0,6024	0,8010
4	1	0,4014	0,4045
4	2	0,4011	0,4031
4	3	0,3994	0,4019

Fonte: elaboração própria, com base nos registros experimentais.

O p95 foi calculado pelo método do posto mais próximo. Como cada execução contém doze consultas, esse percentil corresponde ao maior valor observado. A ocupação dos consumidores permaneceu elevada: entre 99,6% e 99,7% com um consumidor, entre 98,5% e 99,6% com dois e entre 96,7% e 98,2% com quatro.

Os ganhos observados decorrem da possibilidade de sobrepor as esperas simuladas. Por esse motivo, não se pode assumir que o aumento de threads produziria ganho equivalente na inferência de um modelo real limitado por CPU.

Falha controlada e correção

Para demonstrar o comportamento diante da saturação, foi implementado um cenário no qual os produtores tentavam inserir consultas sem aguardar vagas. Quando o buffer estava cheio, a requisição era descartada e registrada. Os consumidores eram iniciados somente após o término dos produtores, garantindo a saturação do buffer.

Na versão corrigida, os produtores aguardavam uma vaga com empty.acquire(). Os consumidores iniciavam após um atraso de 0,5 segundo. Durante esse período, os produtores podiam permanecer bloqueados; após a liberação de vagas, a produção continuava normalmente. Cada cenário foi repetido três vezes.

Tabela 3 — Resultados da falha controlada e da correção

Cenário	Repetições	Enviadas por execução	Processadas por execução	Descartadas por execução	Duplicadas
Falha controlada	3	12	3	9	0
Correção	3	12	12	0	0

Fonte: elaboração própria, com base nos registros experimentais.

A falha consistiu no descarte de consultas que deveriam aguardar atendimento. A espera por vagas preservou todas as requisições nas três execuções corrigidas. Os tempos totais da correção foram de 1,7151 s, 1,7328 s e 1,7162 s, incluindo o atraso inicial dos consumidores.

Como os cenários utilizaram condições diferentes para iniciar os consumidores, seus tempos não foram comparados como medida de ganho de desempenho. A análise concentrou-se na preservação das consultas e no respeito à capacidade do buffer.

Integração com os demais componentes

Após a avaliação isolada, a fila foi integrada aos componentes dos demais integrantes. A ingestão e o armazenamento SQLite foram implementados por Fagner; o cache compartilhado, por Filipe; e a recuperação e geração simulada, por Rauany. A contribuição de Vênisson compreendeu a conexão dessas interfaces à fila, ajustes nos caminhos dos módulos e a validação do atendimento concorrente.

O índice é carregado antes da criação dos consumidores, e a conexão SQLite é fechada após essa etapa. Durante o atendimento, os trabalhadores utilizam os dados em memória e compartilham uma única instância do cache. Quando uma resposta não está disponível no cache, são executadas a recuperação de trechos e a geração simulada.

Foram realizadas três validações com doze requisições por execução.

Tabela 4 — Validação do fluxo integrado

Repetição	Concluídas	Falhas	Duplicações	Acertos do cache	Misses	Tempo do lote (s)

1	12	0	0	9	3	0,0838
2	12	0	0	9	3	0,0843
3	12	0	0	9	3	0,0792

Fonte: elaboração própria, com base nos registros experimentais.

Todas as requisições foram concluídas, com recuperação de contexto do documento armazenado e respeito ao limite do buffer. O contexto recuperado possuía 168 bytes.

Esses tempos excluem a ingestão e o carregamento inicial do índice. Também não são diretamente comparáveis aos experimentos isolados de C3, pois o fluxo integrado utiliza cache e um custo de geração diferente.

Limitações e considerações finais

Os experimentos demonstraram que semáforos e exclusão mútua permitem coordenar produtores e consumidores em um buffer limitado. A espera por vagas evitou os descartes observados na falha controlada, enquanto o aumento dos consumidores melhorou o atendimento da carga simulada.

A avaliação utilizou poucas consultas e três repetições por configuração. Os vetores do índice são simulados por SHA-256; portanto, a busca por cosseno demonstra a execução da recuperação, sem comprovar relevância semântica.

A ordem das impressões no terminal pode diferir da ordem das operações sobre o buffer. Além disso, o encerramento por sentinelas pressupõe o funcionamento normal das threads, sem um timeout global para falhas inesperadas. A coleta identifica o ambiente, os parâmetros e a versão do código, mas não registra individualmente o horário de cada execução do C3.

Os registros brutos estão disponíveis na pasta experimentos/resultados/venisson_c3_20261020_025536/. As validações integradas encontram-se em experimentos/resultados/venisson_integrado/. As instruções de reprodução estão documentadas em sincronizacao/venisson/README.md.

4 – Recuperação e Resposta/Geração – Rauany Ingrid Santos de Jesus

4.1 Objetivo da tarefa

Este componente teve como objetivo implementar a recuperação de trechos relevantes a partir dos documentos previamente processados pelo módulo de ingestão e armazenamento, além da geração de respostas a partir do contexto recuperado. A contribuição compreendeu a integração com o índice SQLite, a seleção dos trechos, a montagem do contexto e a simulação do custo computacional da geração.

A implementação foi desenvolvida em dois módulos independentes: "servidor/recuperacao/__init__.py", responsável pela recuperação e montagem do contexto, e "servidor/resposta/__init__.py", responsável pela geração simulada e pelo registro das métricas de execução. Essa separação permite que os componentes sejam utilizados de forma independente e integrados à fila de requisições e ao cache compartilhado.

4.2 Implementação da recuperação

O módulo de recuperação utiliza a interface "SQLiteIndex.iter_chunk_vectors(batch_size=128)" para acessar os fragmentos armazenados no banco de dados e seus respectivos vetores. Durante a inicialização, os dados do índice são carregados para a memória, evitando o compartilhamento de uma conexão SQLite entre diferentes threads durante o atendimento das consultas.

Para cada consulta, é gerado um vetor determinístico de 128 dimensões, utilizando o mesmo procedimento de vetorização adotado na ingestão. A relevância dos fragmentos é estimada por meio da similaridade do cosseno entre o vetor da consulta e os vetores armazenados. Em seguida, os trechos são ordenados pelo resultado da comparação, e os primeiros k são selecionados conforme o parâmetro "top_k".

O contexto é construído pela concatenação dos textos recuperados, respeitando o limite configurado de 2.000 caracteres. Cada trecho preserva informações de origem, posição e pontuação de similaridade, permitindo relacionar o resultado ao documento indexado.

Entretanto, os vetores utilizados são simulados e determinísticos, produzidos a partir de SHA-256. Embora essa estratégia permita reproduzir as execuções e avaliar o funcionamento do fluxo, ela não representa adequadamente a semântica dos textos. Portanto, a similaridade calculada não comprova que os trechos selecionados sejam semanticamente relevantes para a consulta.

4.3 Implementação da resposta e geração simulada

O módulo de resposta recebe a consulta, realiza a recuperação dos trechos e monta o contexto que será utilizado na geração. Em vez de executar um modelo de linguagem, a implementação simula o custo computacional por meio de um atraso proporcional ao tamanho do contexto, acrescido de uma pequena variação aleatória.

Essa abordagem permite medir o comportamento do componente sem depender da instalação ou da execução de um modelo de linguagem local. A resposta produzida identifica o caráter simulado da geração e apresenta informações resumidas sobre o contexto utilizado.

Durante o processamento, são registradas métricas como tempo de recuperação, tempo de geração, tempo total, quantidade de trechos recuperados e tamanho do contexto. Esses dados permitem comparar diferentes configurações e observar como a quantidade de trechos e o tamanho dos corpora influenciam o tempo de resposta.

4.4 Metodologia experimental

Os experimentos foram executados em ambiente Ubuntu 24.04.5 LTS, utilizando Python 3.12.3, dois processadores lógicos e aproximadamente 3,8 GiB de memória RAM. A coleta foi realizada em 20/10/2026, conforme o registro de execução, utilizando corpora sintéticos de três tamanhos: pequeno, médio e grande.

O benchmark foi executado pelo script "experimentos/scripts/benchmark_recuperacao_geracao.py". Foram utilizadas 15 configurações, com três repetições por configuração, totalizando 45 execuções. Cada execução processou 60 consultas, resultando em 2.700 registros de consulta. Os fatores variados foram o tamanho do corpus, a quantidade de trechos recuperados ("top_k") e o número de trabalhadores utilizados no experimento.

Os resultados foram registrados em arquivos CSV e JSONL, contendo as medições individuais, as estatísticas agregadas e os eventos da coleta. Para a análise, foram considerados o tempo médio, a mediana e o percentil 95 (p95) do tempo total por consulta.

4.5 Resultados e análise

Essa seção apresenta uma seleção dos resultados obtidos, permitindo comparar configurações com diferentes tamanhos de corpus e parâmetros de recuperação.

Tabela 1 – Resultados do benchmark de recuperação e geração

Corpus	"top_k"	Trabalhadores	Média (ms)	Mediana (ms)	p95 (ms)
Pequeno	4	1	314,33	314,23	316,47
Médio	4	1	322,14	321,90	326,49
Grande	4	1	346,85	345,94	359,25

Grande	2	4	266,03	253,22	347,37
Grande	4	4	358,27	349,12	418,86

Fonte: elaboração própria, com base nos registros experimentais.

Na configuração com "top_k=4" e um trabalhador, o tempo médio passou de 314,33 ms no corpus pequeno para 322,14 ms no médio e 346,85 ms no grande. Esse comportamento indica aumento da latência à medida que cresce o conjunto de documentos pesquisados, embora os valores observados não sejam suficientes, isoladamente, para estabelecer uma relação de desempenho válida para qualquer tamanho de corpus.

No corpus grande, a configuração com "top_k=2" e quatro trabalhadores apresentou média de 266,03 ms, inferior aos 358,27 ms registrados com "top_k=4" e quatro trabalhadores. A redução é compatível com o processamento de um contexto menor, mas a comparação não isola completamente o efeito da quantidade de trechos, pois a carga de geração simulada também depende do tamanho do contexto. Além disso, o p95 de 347,37 ms demonstra que a média não descreve sozinha toda a distribuição dos tempos.

A comparação entre um, dois e quatro trabalhadores, mantendo "top_k=4", também não demonstrou ganho consistente com o aumento da concorrência. No corpus médio, as médias foram de 322,14 ms, 322,03 ms e 323,57 ms, respectivamente. No corpus grande, a média passou de 346,85 ms com um trabalhador para 358,27 ms com quatro. Portanto, nas condições avaliadas, o aumento do número de trabalhadores não produziu uma redução consistente da latência individual.

É importante destacar uma ocorrência atípica no corpus pequeno, com "top_k=4" e dois trabalhadores. Uma consulta apresentou duração de 47,0283 segundos, elevando a média da configuração para 833,12 ms, enquanto a mediana permaneceu em 314,12 ms. O p95 registrado foi de 316,43 ms, indicando que a maior parte das consultas teve tempos próximos aos das demais configurações. A causa da ocorrência não foi investigada e, por isso, o valor foi preservado nos resultados, sem ser considerado representativo do comportamento habitual do componente.

4.6 Limitações e considerações finais

Os experimentos demonstraram o funcionamento do fluxo de recuperação, montagem de contexto e geração simulada, além de possibilitarem a comparação de diferentes configurações. Os resultados indicaram que a quantidade de trechos recuperados, o tamanho do contexto e o número de trabalhadores podem influenciar o tempo de resposta, mas que o aumento da concorrência não garante melhoria de desempenho.

A implementação apresenta limitações importantes. Os vetores determinísticos não representam relações semânticas reais entre os textos, e a geração simulada não reproduz o custo de inferência de um modelo de linguagem. Além disso, os tempos apresentados não incluem a espera na fila de requisições, a ingestão dos documentos nem o carregamento inicial do índice. Dessa forma, as medições representam o processamento do componente nas condições específicas do benchmark e não o tempo completo de atendimento de uma aplicação em produção.

Por fim, a contribuição disponibiliza uma interface modular para recuperação e geração, com registro de métricas e integração com o armazenamento SQLite. Os resultados fornecem uma base experimental para analisar o custo de processamento das consultas e compreender a relação entre recuperação de contexto, geração simulada, concorrência e utilização dos recursos computacionais no GenResearch-Servidor.

5 - Cache compartilhado - Filipe de Carvalho Godoy

5.1 Escopo

Implementação e avaliação do componente de cache compartilhado de respostas (`servidor/cache`) e execução do experimento C4, que compara o comportamento do sistema sob consultas repetidas e semanticamente distintas.

O cache guarda respostas recentes indexadas pela consulta, evitando repetir a etapa cara de recuperação e geração. Por ser compartilhado entre trabalhadores concorrentes, todo acesso ao seu estado interno é serializado por um mutex (`threading.Lock`).

5.2 Ambiente de execução

Item	Valor
Sistema	Ubuntu 24.04.5 LTS sobre WSL2
Kernel	6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core i5-10500H — 6 núcleos físicos, 12 threads lógicas
Memória	7,6 GiB · swap 2,0 GiB

Python	3.12.3, apenas biblioteca padrão
Commit	`3b4340d`
Data da coleta	2026-10-18, 23h38 (UTC−03)
Diretório de trabalho	`/home/filipe/gemma`

As medições foram realizadas no sistema de arquivos nativo do Linux, nunca em `/mnt/c`. O acesso ao disco do Windows a partir do WSL atravessa uma camada de tradução que introduz latência adicional e distorceria os tempos medidos.

Ressalva de comparabilidade. Este ambiente difere do utilizado nas configurações C5 e C6 (kernel 6.18, Python 3.14). Tempos absolutos entre configurações medidas em ambientes distintos não são diretamente comparáveis. As comparações internas ao C4 permanecem válidas, pois todas as 36 execuções ocorreram no mesmo ambiente e na mesma sessão.

5.3 Implementação

5.3.1 Estrutura

O componente expõe uma interface única:

from servidor.cache import CacheCompartilhado

cache = CacheCompartilhado(capacidade=32, politica="lru")

resposta, foi_acerto, latencia_s = cache.obter_ou_gerar(consulta, gerador)

O método `obter_ou_gerar` recebe a consulta e uma função geradora. Se a consulta estiver no cache, devolve o valor guardado; caso contrário, chama o gerador, armazena o resultado e o devolve. O `gerador` é qualquer `Callable[[str], Any]`, de modo que o cache não conhece o funcionamento interno da recuperação nem da geração.

Duas políticas de remoção foram implementadas sobre um `OrderedDict`:

- **LRU** (*Least Recently Used*): descarta a entrada usada há mais tempo. A cada acerto, a entrada é movida para o fim da ordem.
- **FIFO** (*First In, First Out*): descarta a entrada inserida há mais tempo, independentemente do uso.

5.3.2 Decisão de projeto: geração fora do mutex

A geração ocorre **fora** da seção crítica. O mutex é adquirido duas vezes: uma para consultar o cache e outra para inserir o resultado. Entre as duas, o lock é liberado.

A alternativa — manter o lock durante toda a operação — garantiria que duas threads nunca gerassem a mesma chave, mas serializaria todos os trabalhadores pelo tempo de geração, anulando a concorrência. Com a geração custando ~15 ms e a seção crítica custando microssegundos, isso reduziria o sistema a um trabalhador efetivo.

O preço da decisão é o **trabalho duplicado**: duas threads podem gerar a mesma chave simultaneamente. Isso produz desperdício, não inconsistência — o resultado final é correto. O custo foi medido: a taxa de acerto caiu de 72,8% (um trabalhador) para 71,2% (quatro), ou seja, 1,6 ponto percentual, em troca de um ganho de 3,6× em vazão.

5.3.3 Contabilidade do tempo economizado

Cada entrada do cache armazena, junto ao valor, o custo que ele teve para ser produzido. A cada acerto, credita-se exatamente esse custo ao tempo economizado.

A alternativa comum — multiplicar o número de acertos por um custo médio estimado — introduziria erro proporcional à variância dos tempos de geração. Como o custo simulado depende do tamanho da consulta, essa variância existe, e a contabilidade exata evita distorção.

5.4 Metodologia experimental

5.4.1 Parâmetros fixados

Conforme exigido em `experimentos/configuracoes.md`, os parâmetros foram fixados e registrados antes da coleta:

Parâmetro	Valor
Consultas por execução	200

Repetições por configuração	3
Semente do gerador aleatório	42 + número da repetição
Universo de consultas distintas (carga repetida)	60
Distribuição de acesso (carga repetida)	enviesada, peso 1/posição
Custo simulado de geração	8 ms + 0,12 ms por caractere + jitter até 4 ms

5.4.2 Fatores variados

Quatro fatores, variados um por vez a partir de uma configuração base (capacidade 32, LRU, 4 trabalhadores), sempre nas duas cargas:

Fator	Valores
Carga	repetidas · distintas
Capacidade do cache	8 · 32 · 128
Política de remoção	LRU · FIFO
Trabalhadores concorrentes	1 · 2 · 4

Total: **12 configurações × 3 repetições = 36 execuções**.

5.4.3 Cargas de trabalho

Repetidas. Universo de 60 consultas distintas, sorteadas com distribuição enviesada (peso 1/posição, aproximando uma Zipf): poucas consultas muito populares e uma cauda longa de consultas raras. Reproduz o padrão de localidade de um serviço real de perguntas.

Distintas. Cada consulta carrega um identificador único, tornando-a irrepetível. Representa exploração aberta, em que o cache não tem como acertar.

5.4.4 Dimensionamento do lote

Cada execução processa 200 consultas, não uma. A decisão decorre do item 10 do enunciado, que exige média, mediana e percentil 95: não é possível calcular um percentil sobre três pontos. O lote por execução fornece a distribuição intra-execução, e as três repetições medem a dispersão entre execuções.

5.4.5 Comandos

execução de conferência

python3 experimentos/scripts/c4_cache.py --validacao --consultas 200 --repeticoes 3

execução registrada

python3 experimentos/scripts/c4_cache.py --consultas 200 --repeticoes 3

A execução de conferência precedeu a registrada e produziu valores dentro da mesma faixa, confirmando a estabilidade das medições. Ambos os conjuntos estão no repositório.

5.5 Resultados

Médias das três repetições da execução registrada.

Carga	Cap.	Pol.	Trab.	Acerto	Subst.	Total (s)	Econ. (s)	Vazão (c/s)	Média (ms)	Mediana (ms)	p95 (ms)
repetidas	8	LRU	4	36,5 %	104	0,48	1,00	421	9,3	13,1	17,3

repetidas	32	LRU	4	71,2%	19	0,22	1,99	911	4,3	0,0	16,5
repetidas	128	LRU	4	73,3%	0	0,21	2,15	963	4,0	0,0	16,4
repetidas	32	FIFO	4	64,5%	28	0,27	1,88	744	5,2	0,0	16,6
repetidas	32	LRU	1	72,8%	22	0,81	2,06	251	4,0	0,0	16,7
repetidas	32	LRU	2	72,5%	21	0,42	2,05	481	4,1	0,0	16,8
distintas	8	LRU	4	0,0%	192	0,80	0,00	251	15,6	15,6	18,4
distintas	32	LRU	4	0,0%	168	0,79	0,00	255	15,5	15,4	18,1
distintas	128	LRU	4	0,0%	72	0,79	0,00	253	15,5	15,6	18,4
distintas	32	FIFO	4	0,0%	168	0,78	0,00	257	15,3	15,4	18,3
distintas	32	LRU	1	0,0%	168	3,12	0,00	64	15,5	15,5	18,5

distintas	32	LRU	2	0,0%	168	1,55	0,00	129	15,4	15,4	18,4

Latência por tipo de evento, agregada sobre as 36 execuções:

Evento	Latência média
Acerto (servido pelo cache)	0,0029 ms
Erro (exigiu geração)	15,12 ms
Razão	**5.135×**

5.5.1 Dispersão dos tempos

Configuração	Média (s)	Desvio-padrão (s)	Mínimo (s)	Máximo (s)
repetidas, cap 32, LRU, 1 trab.	0,806133	0,105749	0,687000	0,888900
repetidas, cap 32, LRU, 2 trab.	0,419800	0,047857	0,366700	0,459600
repetidas, cap 32, LRU, 4 trab.	0,222833	0,032036	0,185900	0,243100
repetidas, cap 8, LRU, 4 trab.	0,475267	0,013996	0,464200	0,491000
repetidas, cap 128, LRU, 4 trab.	0,207933	0,008344	0,198300	0,212900

repetidas, cap 32, FIFO, 4 trab.	0,270700	0,026258	0,240400	0,286800
distintas, cap 32, LRU, 1 trab.	3,115900	0,027042	3,085900	3,138400
distintas, cap 32, LRU, 2 trab.	1,554233	0,012796	1,546400	1,569000
distintas, cap 32, LRU, 4 trab.	0,785800	0,009560	0,779500	0,796800
distintas, cap 8, LRU, 4 trab.	0,797800	0,007238	0,789500	0,802800
distintas, cap 128, LRU, 4 trab.	0,790567	0,007003	0,782800	0,796400
distintas, cap 32, FIFO, 4 trab.	0,777367	0,003530	0,773600	0,780600

A dispersão é sistematicamente maior na carga repetida: desvio-padrão de 0,105 s contra 0,027 s na configuração equivalente da carga distinta. O mecanismo explica a diferença — na carga repetida, o tempo total depende de *quando* cada acerto ocorre, e a ordem de chegada das consultas varia entre repetições. Na carga distinta, toda consulta custa o mesmo (sempre erro), e o tempo total é quase determinístico.

5.6 Análise

5.6.1 O acerto é milhares de vezes mais barato que o erro

Latência média de um acerto: 0,0029 ms. De um erro: 15,12 ms. Razão de 5.135×.

Esse é o fato do qual todos os outros decorrem. O cache não acelera a geração — ele a elimina. Qualquer ganho observado no experimento é proporcional ao número de gerações evitadas, não a alguma otimização do caminho de geração.

5.6.2 A capacidade tem retorno decrescente com ponto de inflexão visível

Passar de 8 para 32 entradas quase dobrou a taxa de acerto, de 36,5% para 71,2%, e reduziu o tempo total de 0,48 s para 0,22 s. Quadruplicar de 32 para 128 rendeu apenas 2,1 pontos percentuais.

O que muda de fato entre 32 e 128 é a substituição: 19 contra zero. Com capacidade 128, o universo inteiro de 60 consultas cabe no cache e nada precisa ser descartado.

A generalização: o ganho de capacidade se esgota quando ela alcança o conjunto de trabalho. Acima desse ponto, aumentar o cache apenas consome memória sem contrapartida.

5.6.3 LRU supera FIFO porque a carga tem localidade

Na mesma capacidade e com os mesmos trabalhadores, LRU alcançou 71,2% contra 64,5% do FIFO — 6,7 pontos percentuais —, com menos substituições (19 contra 28) e vazão maior (911 contra 744 consultas por segundo).

O FIFO descarta por ordem de chegada, sem considerar uso: uma consulta popular inserida cedo é removida mesmo sendo pedida constantemente. O LRU descarta o que não é usado há mais tempo, preservando o conjunto quente.

Condição para a diferença existir. Sob acesso uniforme, as duas políticas empatariam — nenhuma informação sobre uso recente ajudaria a prever o acesso futuro. A vantagem do LRU existe apenas porque a carga tem localidade. Essa dependência é a razão pela qual a carga de trabalho precisou ser corrigida durante o desenvolvimento (seção 5.7.1).

5.6.4 A concorrência escalou quase linearmente

Com capacidade 32 e LRU, o tempo caiu de 0,81 s (1 trabalhador) para 0,42 s (2) e 0,22 s (4), com a vazão subindo de 251 para 481 e 911 consultas por segundo: **3,6× com quatro trabalhadores**.

Na carga distinta a escala foi ainda melhor — 3,12 s para 0,79 s, ou **3,9×**, praticamente o ideal teórico. A diferença tem explicação: na carga repetida boa parte do trabalho são acertos quase instantâneos, de modo que resta menos trabalho paralelizável e o custo fixo pesa proporcionalmente mais.

Duas razões para a ausência de contenção no mutex:

1. **A seção crítica é curta** — consulta a dicionário e atualização de contadores. O trabalho caro acontece fora do lock (seção 5.3.2).
2. **A carga é limitada por espera, não por CPU.** Nesse regime, as threads liberam o GIL durante a espera e são a escolha adequada. Com geração limitada por CPU, o GIL seria o gargalo e processos seriam necessários, com um cache em memória compartilhada.

5.6.5 Na carga distinta o cache é custo puro

Taxa de acerto zero em todas as configurações, tempo economizado zero, e nenhuma variação de tempo total entre capacidades — 0,80 s, 0,79 s e 0,79 s para capacidades 8, 32 e 128. O que muda é apenas o número de substituições: 192, 168 e 72, ou seja, trabalho de manutenção sem contrapartida.

A conclusão não é que o cache seja inútil, e sim que **ele só paga quando há repetição**. Um serviço de perguntas frequentes se beneficia; uma ferramenta de exploração aberta, não. Dimensionar cache sem conhecer o padrão de acesso é adivinhação.

5.6.6 Mediana e p95 contam histórias opostas

Na carga repetida com capacidade 32, a mediana é **0,0 ms** e o p95 é **16,5 ms** — compare com os 18,1 ms de p95 da carga distinta, onde não há acerto nenhum.

A consulta típica é instantânea; a do percentil 95 custa praticamente o mesmo que custaria sem cache algum. O motivo é que a cauda da distribuição *é* o conjunto de erros: o cache move massa para perto de zero, mas não melhora o pior caso — apenas o torna mais raro.

Para dimensionamento por SLA de percentil 95, a distinção é decisiva. A média engana ao ser puxada pelos acertos: 4,3 ms de média, 0,0 ms de mediana e 16,5 ms de p95 descrevem a mesma amostra.

A configuração de capacidade 8 é o caso intermediário instrutivo: mediana de 13,1 ms, entre os dois extremos. Com apenas 36,5% de acerto, menos da metade das consultas é servida pelo cache, e a mediana cai no território dos erros.

5.6.7 Relação com conceitos de Sistemas Operacionais

O comportamento observado reproduz, em nível de aplicação, o que o kernel faz com o cache de páginas:

- Um conjunto de trabalho pequeno em relação à capacidade produz alta taxa de acerto.
- Acesso sem localidade transforma o cache em sobrecarga pura.
- A política de remoção importa exatamente na faixa em que a capacidade é menor que o conjunto de trabalho; fora dela, é irrelevante.
- A razão de 5.135× entre acerto e erro é o análogo da diferença entre servir uma página já residente em memória e buscá-la no disco.

5.7 Falhas identificadas e correções aplicadas

Esta seção registra os problemas encontrados durante o desenvolvimento e as correções correspondentes. Nenhum resultado apresentado na seção 5.5 provém de versões anteriores às correções descritas aqui.

5.7.1 Carga de trabalho inadequada para comparar políticas

Problema. A primeira versão do experimento usava um universo de 20 consultas distintas, sorteadas com distribuição uniforme. Os resultados produzidos foram inconclusivos em dois fatores simultaneamente:

- Capacidades 32 e 128 apresentaram resultados idênticos, porque ambas comportavam o universo inteiro. O fator "capacidade" não discriminava nada acima de 20.
- LRU e FIFO empataram, com diferenças dentro do ruído entre repetições.

Diagnóstico. Sob acesso uniforme, informação sobre uso recente não tem valor preditivo: toda entrada tem a mesma probabilidade de ser acessada em seguida. O LRU perde sua vantagem estrutural sobre o FIFO. O experimento, como estava, não conseguiria responder à pergunta que se propunha a responder.

Correção. O universo foi ampliado para 60 consultas e o sorteio passou a usar distribuição enviesada com peso 1/posição. Isso introduz localidade — poucas consultas muito populares, cauda longa de raras — que é o padrão de cargas reais. Com a correção, a capacidade passou a discriminar (36,5% → 71,2% → 73,3%) e o LRU separou-se do FIFO por 6,7 pontos percentuais.

Lição. O desenho da carga de trabalho determina quais perguntas o experimento consegue responder. Uma carga mal escolhida produz resultados tecnicamente corretos e analiticamente vazios.

5.7.2 Divergência de ambiente entre integrantes

Problema. A instalação padrão do WSL2 forneceu Ubuntu 26.04 com Python 3.14, enquanto o inventário de ambiente do projeto (`ambiente/inventario.md`) registra Ubuntu 24.04.1. As configurações C5 e C6 já haviam sido executadas num terceiro ambiente (kernel 6.18, Python 3.14).

Diagnóstico. O enunciado permite ambientes distintos desde que identificados, mas versões diferentes de Python alteram o desempenho de operações com `threading` e dicionários, e o experimento mede justamente isso. Manter a divergência sem necessidade degradaria a qualidade de qualquer consolidação futura.

Correção. Instalação de uma distribuição Ubuntu 24.04 paralela (`wsl --install -d Ubuntu-24.04`), alinhando ao inventário do projeto. Todas as 36 execuções foram realizadas nesse ambiente. A divergência remanescente em relação a C5 e C6 está registrada explicitamente na seção 5.2.

Lição. Alinhamento de ambiente custa minutos antes da coleta e é irrecuperável depois dela.

5.7.3 Risco de medição contaminada pelo sistema de arquivos

Problema potencial. O WSL2 inicia o terminal em `/mnt/c/...`, que é o disco do Windows montado dentro do Linux. O acesso a esse ponto de montagem atravessa uma camada de tradução de chamadas de sistema, com latência significativamente maior que o sistema de arquivos nativo.

Correção preventiva. Todo o trabalho foi realizado em `/home/filipe/gemma`. Embora o experimento de cache seja predominantemente limitado por CPU e espera — e portanto menos sensível ao sistema de arquivos que C1 e C2 —, a escrita do CSV e do log JSONL ocorre em disco, e a precaução elimina uma variável desnecessária.

5.7.4 Validação por execução dupla

Procedimento adotado. Em vez de uma única coleta, o experimento foi executado duas vezes: uma de conferência (`--validacao`, gravando em `c4_validacao.csv`) e uma registrada (`c4_cache_compartilhado.csv`). Os valores das duas ficaram dentro da mesma faixa — por exemplo, 71,2% contra 71,2% de taxa de acerto na configuração base, e 0,2228 s contra 0,2213 s de tempo médio.

A convergência confirma que os resultados não dependem de condições transitórias da máquina no momento da coleta.

5.8 Limitações

- **A geração é simulada.** O custo de recuperação e geração é reproduzido por um atraso proporcional ao tamanho da consulta, alternativa autorizada pelo enunciado (Parte B, "Resposta"). Os tempos absolutos não representam o custo real; as comparações relativas entre configurações permanecem válidas, pois todas usam o mesmo gerador. A integração com `servidor/recuperacao` e `servidor/resposta` exige apenas passar a função real como segundo argumento de `obter_ou_gerar` — nenhuma outra alteração é necessária.
- **A chave do cache é a consulta exata.** Consultas semanticamente equivalentes com redação diferente contam como entradas distintas. O módulo oferece normalização opcional (minúsculas, colapso de espaços, remoção de pontuação de borda), não utilizada nestas medições. Acerto por similaridade semântica exigiria comparação de embeddings, fora do recorte adotado.
- **Ambiente distinto do usado em C5 e C6**, conforme registrado na seção 5.2.
- **Threads, não processos.** Adequado para carga limitada por espera. Com geração limitada por CPU, o GIL seria o gargalo.
- **Três repetições por configuração** atendem ao mínimo do enunciado, mas são poucas para conclusões estatísticas robustas. O desvio-padrão de 0,105 s na configuração de um trabalhador, sobre média de 0,806 s, indica variabilidade de aproximadamente 13%.

- **Não foram medidos RSS, VSZ nem faltas de página.** O cache mantém no máximo 128 respostas curtas em memória, volume irrelevante frente aos 7,6 GiB disponíveis. A análise de pressão de memória é objeto de C5.
- **O experimento não é integrado ao pipeline completo.** Mede o componente de cache isoladamente, com gerador simulado. A execução integrada com fila, recuperação e resposta reais permanece pendente.

5.9 Contribuição para a consolidação C1–C6

Configuração	Descrição	Situação
C4	Utilização de cache	Resultados coletados — 12 configurações × 3 repetições

Resumo para a comparação geral: na carga com repetição e capacidade adequada, o cache elevou a taxa de acerto a 71,2% e reduziu o tempo total de 0,79 s para 0,22 s — **3,6× mais rápido** que a mesma carga sem acertos possíveis, com vazão subindo de 255 para 911 consultas por segundo. Na carga sem repetição, não houve ganho algum.

As estatísticas de dispersão estão em `experimentos/resultados/c4_estatisticas.csv`, no mesmo formato adotado em C5 e C6.

5.10 Arquivos de evidência

Arquivo	Conteúdo
`servidor/cache/cache.py`	Implementação do cache compartilhado
`servidor/cache/__init__.py`	Interface pública do pacote
`servidor/cache/README.md`	Contrato de integração e decisões de projeto

`experimentos/scripts/c4_cache.py`	Script do experimento
`experimentos/analise_c4.md`	Análise detalhada
`experimentos/resultados/c4_cache_compartilhado.csv`	36 execuções, uma por linha
`experimentos/resultados/c4_estatisticas.csv`	Média, desvio-padrão, mínimo e máximo
`experimentos/resultados/c4_validacao.csv`	Execução de conferência
`experimentos/resultados/c4_validacao_estatisticas.csv`	Estatísticas da conferência
`experimentos/resultados/c4_eventos.jsonl`	Log de eventos com escrita concorrente protegida

6 - Leitura convencional vs. mmap + Memória Virtual (C1 + C2 + Parte E) — Giulian

Giulian Fabio Bastos Amorim Lima

Objetivo:

Comparar a leitura convencional de arquivos com `open()` e `read()`e o acesso a arquivos por mapeamento de memória usando `mmap`,observando desempenho, consumo de memória e faltas de página.

- Corpus pequeno: arquivos de 64 KiB.

- Corpus grande: arquivos de 8 MiB.

- Repetições: 3 por arquivo e método com 2 passagens.

- Tamanho do bloco de leitura convencional: 65.536 bytes.

Leitura convencional:

O programa abre o arquivo com `open()` e lê seu conteúdo em blocos usando `read()`.

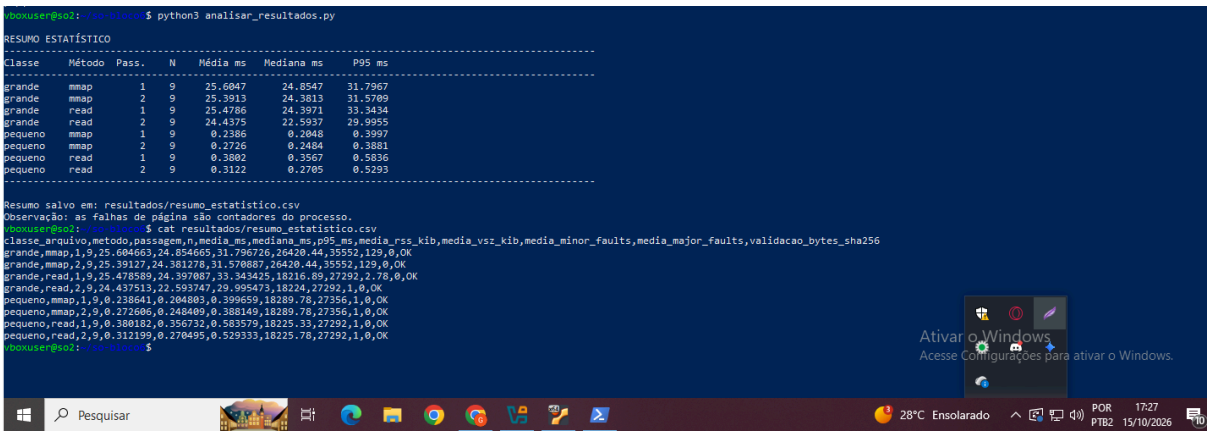
Mapeamento de memória:

O programa abre o arquivo e utiliza `mmap` para mapear seu conteúdo na memória virtual, acessando os dados em blocos.

Estrutura do teste:

```
vboxuser@so2:~/so-bloco6$ python3 --version
Python 3.12.3
vboxuser@so2:~/so-bloco6$ nano gerar_corpus.py
vboxuser@so2:~/so-bloco6$ vboxuser@so2:~/so-bloco6$ python3 gerar_corpus.py
usage: gerar_corpus.py [-h] [--corpus CORPUS] [--repeticoes REPETICOES] [--bloco BLOCO] [--saida SAIDA]
gerar_corpus.py: error: Nenhum arquivo encontrado em /home/vboxuser/so-bloco6/corpus
vboxuser@so2:~/so-bloco6$ nano gerar_corpus.py
vboxuser@so2:~/so-bloco6$ python3 gerar_corpus.py
Criado: /home/vboxuser/so-bloco6/corpus/pequenos/documento_1.txt
Tamanho: 65536 bytes
Criado: /home/vboxuser/so-bloco6/corpus/grandes/documento_1.txt
Tamanho: 8388608 bytes
Criado: /home/vboxuser/so-bloco6/corpus/pequenos/documento_2.txt
Tamanho: 65536 bytes
Criado: /home/vboxuser/so-bloco6/corpus/grandes/documento_2.txt
Tamanho: 8388608 bytes
Criado: /home/vboxuser/so-bloco6/corpus/pequenos/documento_3.txt
Tamanho: 65536 bytes
Criado: /home/vboxuser/so-bloco6/corpus/grandes/documento_3.txt
Tamanho: 8388608 bytes
```

Resultados observados:



Nos arquivos pequenos, o mmap apresentou menor tempo médio nas duas passagens. Nos arquivos grandes, a leitura convencional apresentou menor tempo médio, com diferenças relativamente pequenas.

Nas medições dos arquivos grandes, o mmap apresentou RSS e VSZ maiores. Isso é compatível com o mapeamento do arquivo, mas VSZ não representa diretamente a memória física ocupada.

A validação realizada comparou 36 pares de registros. Não foram encontradas diferenças na quantidade de bytes processados nem no SHA-256 entre os métodos.

Limitações:

- Os tempos incluem o processamento necessário para calcular SHA-256.

- As faltas de página são diferenças observadas no processo e não

  representam exclusivamente as faltas causadas pelo arquivo.

- A primeira passagem não garante cache de disco frio.

- O resultado vale para o ambiente, corpus e parâmetros utilizados;

  não permite concluir que um método seja sempre superior.

Conclusão:

O experimento demonstra que o desempenho de read() e mmap depende do tamanho dos arquivos e das condições de execução. Para o corpus pequeno testado, mmap apresentou menor tempo médio. Para o corpus grande, read() apresentou uma pequena vantagem. A escolha entre os métodos deve considerar o padrão de acesso, a memória disponível, a complexidade de implementação e o perfil da aplicação.

Os dois métodos acessam os dados de maneiras diferentes. O read() copia os dados para um buffer do programa, enquanto o mmap permite acessá-los por meio da memória virtual, podendo evitar essa cópia adicional.

Isso pode influenciar o tempo de leitura, mas também depende do tamanho do arquivo e do cache do sistema operacional. Por isso, a equipe fez três repetições para comparar os resultados com mais confiabilidade e evitar depender de uma única execução.

7 - Katyane dos Santos

Realizar e documentar os experimentos correspondentes às configurações **C1 e C2**, avaliando o comportamento do modelo **google/gemma-4-E2B-it-qat-q4_0-gguf** durante a execução local com Ollama.

A configuração **C1** representa o cenário de referência (*baseline*), no qual apenas uma requisição é executada por vez. A configuração **C2** mantém o mesmo modelo, prompts e ambiente experimental, porém realiza **três requisições simultâneas**, permitindo analisar o impacto da concorrência sobre o tempo de resposta e o consumo de recursos computacionais.

Também fizeram parte da tarefa a coleta e comparação das métricas de **TTFT (Time to First Token)**, tempo total de execução, quantidade de tokens gerados, taxa de geração em tokens por segundo, além do monitoramento complementar de **CPU, memória RAM e número de threads**.

6.1 Ambiente experimental

Os experimentos foram realizados em um computador com **Windows 11**, utilizando o **WSL2** como ambiente Linux.

A configuração utilizada foi:

- Processador: Intel Core i5-12450H;
- 8 núcleos e 12 processadores lógicos no equipamento;
- Limite configurado para o WSL2: **8 processadores lógicos**;
- Memória destinada ao WSL2: **5 GB de RAM**;
- Swap configurada: **4 GB**;
- Sistema no WSL2: Ubuntu 26.04.1 LTS;
- Ollama: versão 0.34.0;
- Open WebUI executado através do Docker Desktop;

- Modelo: google/gemma-4-E2B-it-qat-q4_0-gguf;
- Modelo no Ollama: hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest;
- Quantização: Q4_0;
- Tamanho aproximado do modelo: 4,3 GB;
- Contexto utilizado: 4096 tokens;
- Execução do modelo realizada em CPU.

O Open WebUI foi configurado para acessar o Ollama através da porta local 11434. Durante os ajustes da interface, o modo de raciocínio foi desativado e a chamada de função foi configurada como **Legado**, de forma a evitar que o conteúdo enviado ultrapassasse o limite de contexto disponível.

6.2 Metodologia dos experimentos

Para manter a comparação entre C1 e C2, foram utilizados o mesmo modelo, ambiente, quantização e prompts. A principal variável modificada foi o número de requisições executadas simultaneamente.

Foram utilizadas duas cargas de trabalho:

Carga pequena

Prompt:

> Explique em 3 frases o que é um sistema operacional.

A entrada apresentou **22 tokens de prompt**.

Carga grande

Prompt:

> Explique detalhadamente como um sistema operacional gerencia os recursos de um computador. Na resposta, aborde os seguintes aspectos: gerenciamento de processos, escalonamento da CPU, threads, memória principal, memória virtual, sistema de arquivos, dispositivos de entrada e saída, interrupções, chamadas de sistema e mecanismos de proteção e segurança. Explique também como esses componentes se relacionam durante a execução simultânea de vários programas. Organize a explicação de maneira didática, utilizando aproximadamente 10 parágrafos.

A entrada apresentou **120 tokens de prompt**.

Para cada configuração foram realizadas **duas repetições com a carga pequena e duas repetições com a carga grande**.

Antes de cada repetição oficial, o modelo era descarregado utilizando o comando:

ollama stop hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest

Esse procedimento foi utilizado para que cada repetição começasse com o modelo fora da memória, incluindo o custo de carregamento inicial.

As requisições foram enviadas diretamente para a API local do Ollama:

http://127.0.0.1:11434/api/generate

Foi utilizado stream=true, permitindo medir o momento em que o primeiro fragmento de resposta era recebido.

O **TTFT** foi calculado como o intervalo entre o envio da requisição e o recebimento do primeiro conteúdo gerado pelo modelo.

A taxa de geração foi calculada a partir de:

tokens/s = eval_count / (eval_duration / 1.000.000.000)

6.3 Configuração C1 — Baseline

Na configuração C1 foi executada **uma requisição por vez**, representando o cenário de referência para comparação com a execução concorrente.

C1 — carga pequena

Repetição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	84,667 s	103,863 s	81	6,573
2	73,574 s	78,705 s	76	14,959

O TTFT médio da carga pequena foi de aproximadamente:

79,12 segundos.

C1 — carga grande

Repetição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	92,794 s	195,142 s	1518	14,991
2	86,351 s	163,459 s	1061	13,952

O TTFT médio da carga grande foi:

89,57 segundos.

Os resultados mostram que a carga grande exigiu maior tempo total de execução devido principalmente à maior quantidade de tokens gerados.

6.4 Configuração C2 — Três requisições simultâneas

Na configuração C2 foram enviadas **três requisições ao mesmo tempo**.

Foi utilizado um script Python com:

ThreadPoolExecutor(max_workers=3)

Cada repetição da C2 corresponde, portanto, a um lote contendo três solicitações concorrentes.

C2 — carga pequena

Repetição 1

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	73,953 s	78,216 s	85	16,383

2	81,786 s	85,847 s	74	18,387
3	78,362 s	81,642 s	60	18,641

Tempo total do lote: **85,917 s**.

Total gerado: **219 tokens**.

Repetição 2

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	71,605 s	76,311 s	73	18,683
2	63,737 s	71,467 s	80	13,863
3	76,627 s	80,857 s	74	17,024

Tempo total do lote: **80,874 s**.

Total gerado: **227 tokens**.

Na carga pequena, a diferença em relação à configuração C1 foi relativamente limitada.

C2 — carga grande

Repetição 1

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s

1	232,720 s	300,166 s	1108	16,470
2	93,797 s	165,570 s	1119	15,787
3	165,753 s	232,506 s	1085	16,311

Tempo total do lote: **300,189 s**.

Total gerado: **3312 tokens**.

Repetição 2

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	216,354 s	284,914 s	1134	16,594
2	157,940 s	216,186 s	972	16,753
3	83,400 s	157,752 s	1171	15,892

Tempo total do lote: **284,947 s**.

Total gerado: **3277 tokens**.

Considerando as seis requisições da carga grande, o TTFT médio da C2 foi de aproximadamente:

158,33 segundos.

6.5 Comparação entre C1 e C2

A maior diferença entre as configurações apareceu na carga grande.

Métrica	C1	C2
Requisições simultâneas	1	3
TTFT médio — carga grande	89,57 s	158,33 s
CPU média — monitoramento grande	150,31%	231,00%
RAM média — monitoramento grande	3,38 GB	3,39 GB
Máximo de threads	15	15

O TTFT passou de **89,57 s para 158,33 s**, representando um aumento de aproximadamente:

76,8%.

Esse resultado indica que, no ambiente analisado, permitir três solicitações simultâneas aumentou a concorrência pelos recursos computacionais e prejudicou a latência individual das requisições na carga mais pesada.

6.6 Monitoramento de CPU, RAM e threads

Além das repetições oficiais, foram realizadas execuções adicionais destinadas especificamente ao monitoramento dos recursos utilizados pelo processo de inferência.

É importante destacar que essas medições **não correspondem exatamente às mesmas execuções utilizadas nas tabelas de TTFT**, sendo utilizadas como evidência complementar do comportamento do sistema.

Configuração	CPU média	Pico CPU	RAM média	Pico RAM	Threads médias	Máximo
C1 pequena	107,32%	137%	3195,68 MB	3991,66 MB	13,37	15
C1 grande	150,31%	227%	3381,68 MB	4039,43 MB	13,93	15
C2 pequena	104,03%	141%	3107,00 MB	3956,31 MB	13,38	15
C2 grande	231,00%	310%	3387,51 MB	3938,38 MB	14,49	15

O resultado mais significativo foi observado na CPU durante a carga grande.

A média passou de aproximadamente **150,31% na C1 para 231% na C2**.

Em Linux, um processo multithread pode apresentar utilização acima de 100%, pois esse valor representa o uso agregado de múltiplos processadores lógicos. Portanto, 231% corresponde aproximadamente ao equivalente à utilização de 2,31 processadores lógicos.

Em contrapartida, a memória RAM praticamente não aumentou proporcionalmente ao número de requisições.

Isso ocorre porque as requisições compartilham os pesos do mesmo modelo carregado em memória, em vez de carregar três cópias completas do modelo.

Q7 — Qual foi o impacto da concorrência no desempenho e no uso de recursos?

A configuração C2 permitiu que três requisições fossem submetidas simultaneamente, mas isso não resultou em melhora da responsividade individual.

Na carga pequena, os resultados permaneceram relativamente próximos aos observados na C1. Entretanto, na carga grande, o TTFT médio aumentou de **89,57 segundos para 158,33 segundos**, aproximadamente **76,8% de aumento**.

Também foi observado aumento relevante no consumo de CPU durante as execuções monitoradas, passando de uma média de **150,31% na C1 grande para 231% na C2 grande**.

Assim, os experimentos demonstram um *trade-off* entre concorrência e latência. A concorrência permite manter várias solicitações em andamento, porém, em um ambiente limitado por CPU e memória, as requisições passam a disputar os mesmos recursos computacionais.

Q8 — Quais recursos do sistema operacional passaram a ser mais disputados?

Durante a execução concorrente, a principal competição ocorreu pelo tempo de processamento da **CPU**.

As requisições também disputam recursos relacionados à memória, largura de banda de memória, estados de contexto do modelo e estruturas internas utilizadas durante a inferência.

O ambiente utilizado possuía apenas **5 GB de RAM destinados ao WSL2**, enquanto o modelo ocupava aproximadamente **4,3 GB em armazenamento e cerca de 3 a 4 GB de memória durante a execução**. Dessa forma, a utilização de **swap** também se tornou relevante para permitir a execução quando a pressão de memória aumentava.

Entretanto, a RAM utilizada não triplicou ao passar de uma para três requisições porque os pesos do modelo permanecem compartilhados.

Portanto, os resultados indicam que a concorrência aumentou principalmente a pressão sobre a **CPU**, enquanto memória e threads permaneceram relativamente próximas entre C1 e C2.

Dificuldades encontradas

Durante a realização dos experimentos ocorreram algumas dificuldades relacionadas às limitações do equipamento e à configuração do ambiente.

O computador utilizado possui **8 GB de memória RAM física**, o que exigiu limitar cuidadosamente os recursos destinados ao WSL2. Foram configurados **5 GB de RAM e 4 GB de swap**.

Outro problema ocorreu inicialmente no Open WebUI, quando uma requisição excedeu o contexto máximo de **4096 tokens**. O problema foi contornado mantendo o contexto em 4096, desativando o

modo de raciocínio e configurando a chamada de função como **Legado**, reduzindo a quantidade de tokens adicionais gerados pela interface.

Também foi observado que o carregamento do modelo após sua remoção da memória apresentava tempo elevado. Entretanto, esse comportamento foi mantido intencionalmente nos experimentos oficiais para que todas as repetições fossem realizadas utilizando a mesma metodologia de inicialização a frio.

Conclusão dos experimentos C1 e C2

Os experimentos permitiram comparar o comportamento do modelo Gemma nas configurações C1 e C2.

A configuração C1 apresentou melhor latência individual, especialmente na carga grande. Já a C2 possibilitou a submissão de três solicitações simultaneamente, porém apresentou maior competição pelos recursos computacionais disponíveis.

O impacto tornou-se mais evidente no cenário de maior carga, no qual o TTFT médio aumentou aproximadamente **76,8%** e o uso médio de CPU nas execuções monitoradas passou de **150,31% para 231%**.

Por outro lado, a utilização de memória permaneceu próxima entre as duas configurações, indicando o compartilhamento dos pesos do modelo entre as requisições.

Dessa forma, os resultados evidenciam a existência de um compromisso entre **concorrência, latência e utilização de recursos**. Em equipamentos com memória e capacidade de processamento limitadas, aumentar o número de solicitações simultâneas pode permitir atender múltiplos usuários, mas pode elevar consideravelmente o tempo de resposta individual.

8 - Conclusão para o Relatório

A realização deste trabalho permitiu analisar a execução local de um modelo de linguagem não apenas como uma aplicação de inteligência artificial, mas como um sistema diretamente dependente dos mecanismos e recursos administrados pelo sistema operacional. A preparação dos ambientes com Linux, Ollama e Open WebUI possibilitou executar localmente o modelo Gemma quantizado e evidenciou a importância de recursos como CPU, memória RAM, swap e virtualização para a viabilidade e o desempenho da inferência. Os diferentes ambientes utilizados pelo grupo também

mostraram que a disponibilidade e a alocação desses recursos influenciam diretamente o comportamento da aplicação.

A observação da aplicação em funcionamento permitiu relacionar conceitos estudados em Sistemas Operacionais com uma carga de trabalho real. Foi possível identificar a hierarquia entre o serviço Ollama e o processo llama-server, observar a utilização de múltiplas threads durante a inferência e acompanhar estados e consumo de recursos. Dessa forma, processos, threads, concorrência e gerenciamento de recursos deixaram de ser apenas conceitos teóricos e puderam ser observados no funcionamento de uma aplicação contemporânea de IA generativa.

Os experimentos também demonstraram que não existe uma configuração ideal independente do hardware disponível. Na comparação entre C1 e C2, o aumento da concorrência permitiu processar múltiplas solicitações simultaneamente, mas aumentou a disputa por recursos e a latência individual; na carga maior, o TTFT médio aumentou aproximadamente 76,8% e o uso médio de CPU passou de 150,31% para 231%. Na configuração C3, o aumento da janela de contexto de 1024 para 4096 tokens elevou o tempo total médio de 408,967 s para 459,583 s, reduziu a velocidade média de geração e apresentou uma ocorrência de timeout. Esses resultados evidenciam um compromisso entre concorrência, tamanho de contexto, latência, estabilidade e utilização dos recursos computacionais.

Por fim, a execução local com Ollama e Open WebUI oferece maior autonomia e controle sobre o modelo, suas configurações e os dados processados, além de reduzir a necessidade de enviar prompts para serviços externos. Entretanto, esse controle também transfere ao usuário responsabilidades relacionadas à segurança, manutenção, configuração e dimensionamento do hardware. Assim, o trabalho demonstrou que a execução local de modelos de linguagem é viável, mas seu desempenho e sua disponibilidade dependem diretamente das decisões de configuração e dos recursos administrados pelo sistema operacional.

8 – Pressão de Memória, Falha Controlada e Consolidação dos Experimentos (C5 + C6) **Responsável:** Katyane dos Santos

8.1 Objetivo da tarefa

A atividade teve como objetivo analisar o comportamento do sistema diante do aumento gradual da utilização de memória e de situações de falha controlada, relacionando os resultados aos conceitos estudados na disciplina de Sistemas Operacionais.

A responsabilidade foi dividida em três partes principais:

- **Configuração C5 – Pressão de memória:** avaliar o aumento do consumo de memória, monitorando tempo de execução, memória residente, memória virtual, faltas de página e utilização de swap.
- **Configuração C6 – Falha controlada:** simular uma situação de erro durante o acesso a um arquivo e verificar se o programa consegue identificar e tratar a falha adequadamente.
- **Consolidação dos experimentos:** organizar os resultados disponíveis das configurações C1 a C6, incluindo métricas, análises estatísticas e gráficos.

Os experimentos C5 e C6 foram realizados por meio de scripts Python independentes, utilizando cargas e situações simuladas de maneira controlada.

8.2 Ambiente experimental

Os testes foram executados em um computador com Windows 11, utilizando o Ubuntu por meio do WSL2 (Windows Subsystem for Linux).

O ambiente de execução apresentava as seguintes características:

Recurso	Configuração
Sistema operacional principal	Windows 11
Ambiente Linux	Ubuntu no WSL2
Linguagem	Python 3.14.4
Memória disponível ao WSL2	Aproximadamente 4,8 GiB
Swap configurada	4 GiB

Ambiente virtual Python	`.venv`
Controle de versão	Git e GitHub

O uso do WSL2 permitiu observar métricas disponibilizadas pelo Linux, como informações de memória e contadores de faltas de página.

Os experimentos foram executados com cargas limitadas para reduzir o risco de esgotamento da memória disponível.

8.3 Configuração C5 – Pressão de memória

8.3.1 Metodologia

O experimento C5 teve como objetivo observar como o aumento gradual da carga de memória influencia o consumo de recursos e o tempo de execução.

Para isso, foi desenvolvido o script:

`experimentos/scripts/c5_pressao_memoria.py`

O programa realiza a alocação de blocos de memória utilizando a estrutura `bytearray` do Python e acessa as páginas alocadas para provocar a utilização efetiva da memória física.

Foram utilizadas três cargas de trabalho:

- Carga pequena: 100 MiB;
- Carga intermediária: 250 MiB;
- Carga grande: 500 MiB.

Cada carga foi executada três vezes, totalizando nove medições oficiais.

Durante os testes, foram coletadas as seguintes métricas:

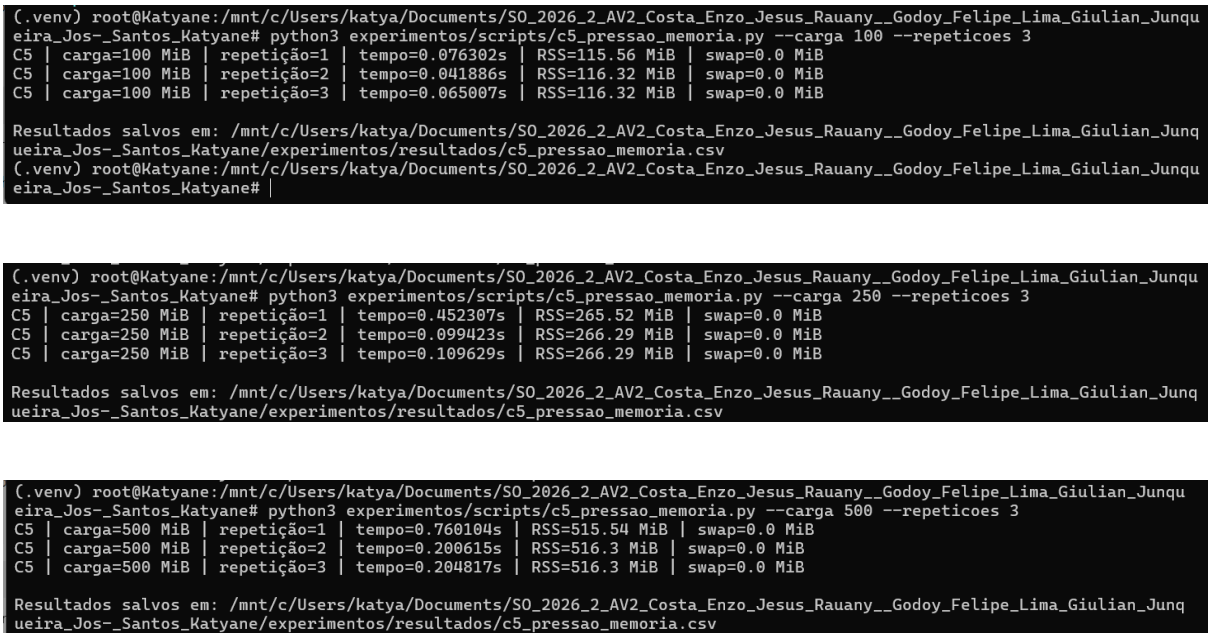
- Tempo de execução da operação;
- RSS (Resident Set Size), que representa a memória física residente utilizada pelo processo;
- VSZ (Virtual Size), correspondente ao espaço de endereçamento virtual;
- MemAvailable, que estima a memória disponível para novos processos;
- Faltas menores e maiores de página;

- Utilização de swap.

A escolha das cargas permitiu observar o crescimento do consumo de memória sem utilizar volumes excessivos que pudessem comprometer a estabilidade do computador.

8.3.2 Evidência da execução

Figura 1 – Execução do experimento C5 no terminal Ubuntu.



Fonte: Elaborado pela autora, a partir da execução do script c5_pressao_memoria.py.

O registro da execução deve demonstrar as cargas utilizadas e os resultados coletados pelo programa.

8.3.3 Resultados obtidos

Os resultados médios das três repetições para cada carga estão apresentados na tabela.

Tabela 1 – Resultados do experimento C5

Carga de memória	Repetições	Tempo médio (s)	RSS médio (MiB)	Swap utilizada
100 MiB	3	0,138452	116,20	0 MiB
250 MiB	3	0,280650	266,21	0 MiB

500 MiB	3	0,554763	516,18	0 MiB

Observa-se que o aumento da carga de memória provocou crescimento tanto no tempo de execução quanto na quantidade de memória física utilizada pelo processo.

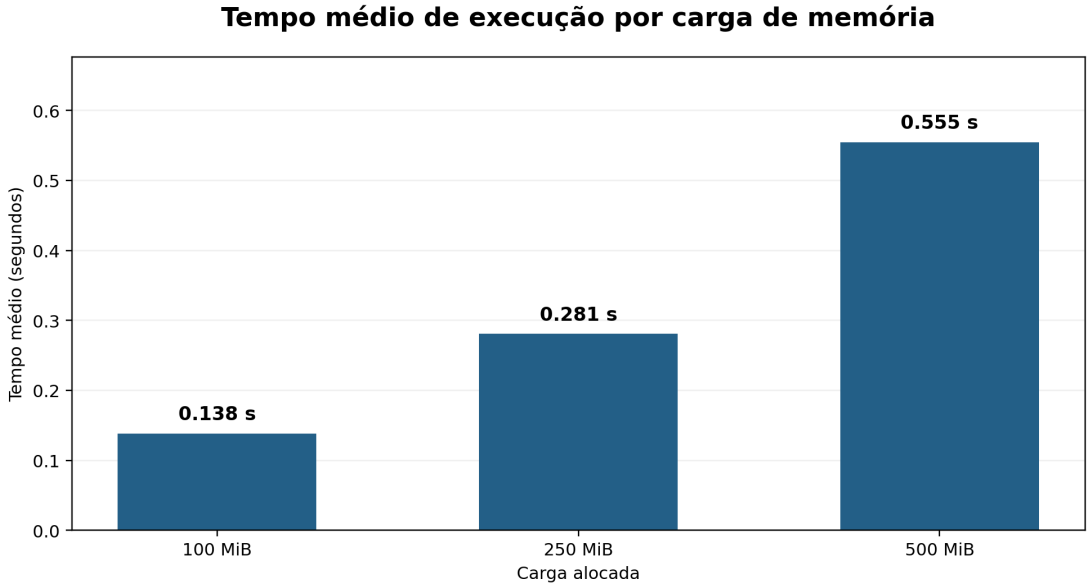
Na carga de 100 MiB, o tempo médio foi de aproximadamente 0,138 segundo, com RSS médio de 116,20 MiB.

Na carga de 250 MiB, o tempo médio aumentou para aproximadamente 0,281 segundo, enquanto o RSS médio atingiu 266,21 MiB.

Na carga de 500 MiB, foi observado o maior consumo de memória, com RSS médio de 516,18 MiB e tempo médio de aproximadamente 0,555 segundo.

8.3.4 Análise dos gráficos

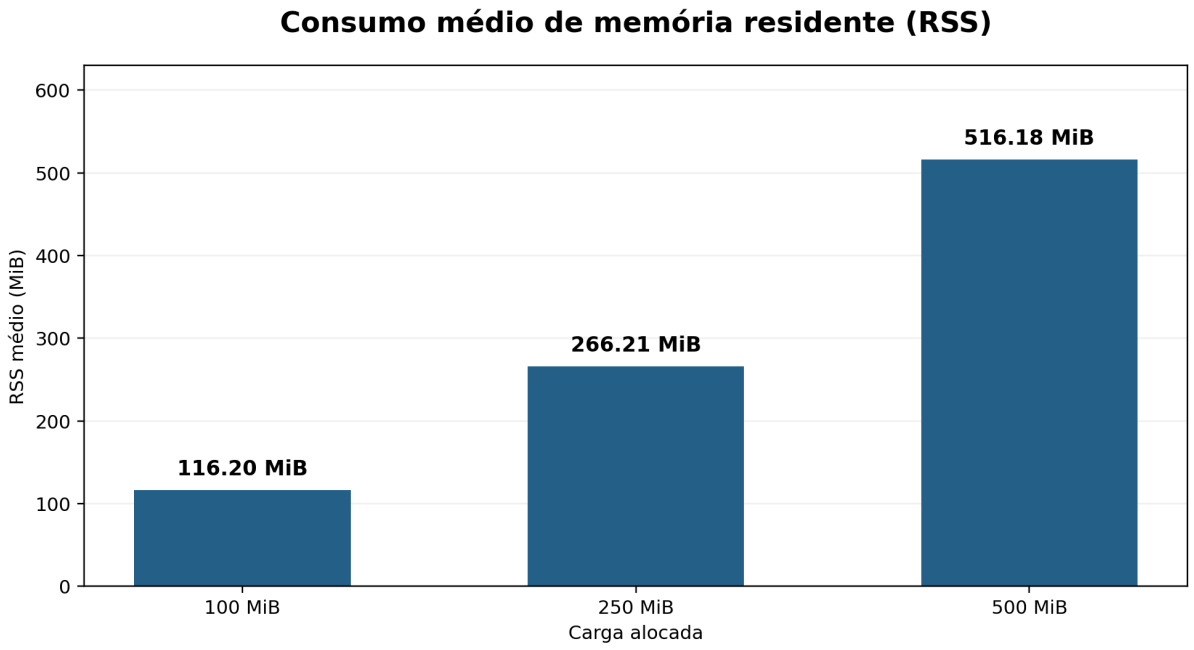
Figura 2 – Tempo médio de execução por carga de memória.



Fonte: Elaborado pela autora com base nos resultados do experimento C5.

O gráfico demonstra uma tendência de crescimento do tempo de execução conforme aumenta a quantidade de memória alocada.

Figura 3 – Consumo médio de memória RSS por carga.



Fonte: Elaborado pela autora com base nos resultados do experimento C5.

O segundo gráfico evidencia o crescimento do consumo de memória física do processo, acompanhando o aumento das cargas de trabalho.

8.3.5 Relação com Sistemas Operacionais

O experimento C5 está diretamente relacionado ao gerenciamento de memória realizado pelo sistema operacional.

Ao solicitar novas regiões de memória, o processo utiliza mecanismos de memória virtual. Durante o acesso às páginas, o sistema operacional pode precisar estabelecer o mapeamento dessas páginas para a memória física.

Nos testes realizados, foram registradas faltas menores de página, que podem ocorrer quando o sistema precisa resolver o acesso a uma página sem realizar leitura de dados do disco.

Não foram registradas faltas maiores de página nem utilização de swap durante as execuções avaliadas.

Embora o ambiente possuísse swap configurada, isso não significa que ela necessariamente seria utilizada. O uso depende da pressão real sobre a memória e das decisões do sistema operacional.

Assim, o experimento permitiu observar o crescimento do consumo de memória, mas não atingiu uma situação de pressão suficiente para demonstrar paginação em disco ou degradação associada ao uso de swap.

8.4 Configuração C6 – Falha controlada

8.4.1 Metodologia

O experimento C6 teve como objetivo verificar o comportamento do programa diante de uma situação de falha previsível.

Foi desenvolvido o script:

`experimentos/scripts/c6_falha_controlada.py`

O cenário escolhido foi a tentativa de abertura de um arquivo inexistente.

Durante a execução, o programa tenta acessar o arquivo:

`experimentos/dados/arquivo_inexistente.txt`

Como o arquivo não existe, o Python produz a exceção `FileNotFoundError`.

O programa utiliza tratamento de exceções para identificar essa situação e registrar o resultado sem provocar um encerramento inesperado.

Foram realizadas três execuções oficiais.

8.4.2 Evidência da execução

Figura 4 – Execução do experimento C6 no terminal Ubuntu.

```
(.venv) root@Katyane:/mnt/c/Users/katya/Documents/SO_2026_2_AV2_Costa_Enzo_Jesus_Rauany__Godoy_Felipe_Lima_Giulian_Junqu
eira_Jos-_Santos_Katyane# python3 experimentos/scripts/c6_falha_controlada.py --arquivo experimentos/dados/arquivo_inexi
stente.txt --repeticoes 3
C6 | repeticao=1 | status=falha_tratada | erro=FileNotFoundError | tempo=0.004523s
C6 | repeticao=2 | status=falha_tratada | erro=FileNotFoundError | tempo=0.001803s
C6 | repeticao=3 | status=falha_tratada | erro=FileNotFoundError | tempo=0.005167s

Resultados salvos em: /mnt/c/Users/katya/Documents/SO_2026_2_AV2_Costa_Enzo_Jesus_Rauany__Godoy_Felipe_Lima_Giulian_Junq
ueira_Jos-_Santos_Katyane/experimentos/resultados/c6_falha_controlada.csv
```

Fonte: Elaborado pela autora, a partir da execução do script c6_falha_controlada.py.

A evidência deve demonstrar que o arquivo não foi encontrado e que o programa registrou a falha como tratada.

8.4.3 Resultados obtidos

Tabela 2 – Resultados do experimento C6

Métrica	Resultado
Tipo de falha	Arquivo inexistente
Exceção identificada	`FileNotFoundError`
Número de execuções	3
Falhas tratadas	3
Tempo médio	0,002523 s
Encerramento inesperado	Não observado

Todas as execuções produziram a falha esperada, que foi identificada e tratada pelo programa.

O tempo médio registrado foi de aproximadamente 0,002523 segundo.

8.4.4 Análise dos resultados

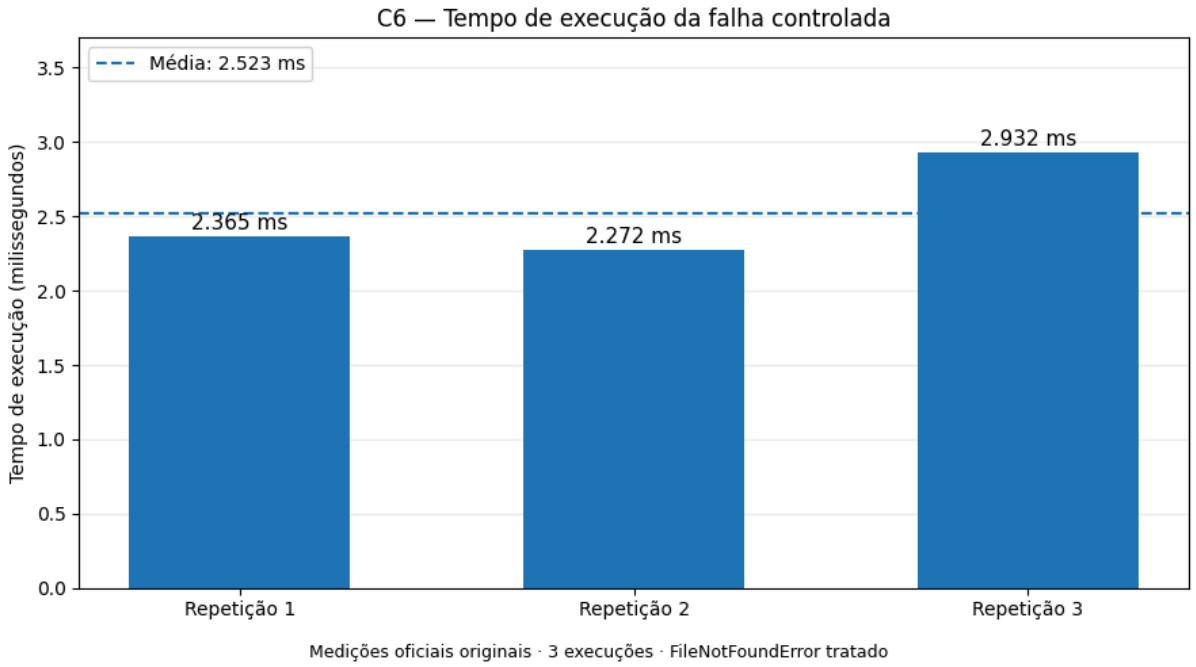
Os resultados demonstraram que o programa conseguiu reconhecer a ausência do arquivo e executar o tratamento previsto.

Esse comportamento é importante em aplicações que dependem de arquivos, bancos de dados, índices ou outros recursos externos.

Sem tratamento adequado, uma falha de acesso pode interromper uma operação e comprometer o funcionamento da aplicação.

O tratamento de exceções permite registrar o problema e decidir como o programa deve continuar ou encerrar a operação de forma controlada.

Figura 5 – Tempo registrado nas execuções da falha controlada.



Fonte: Elaborado pela autora com base nos resultados do experimento C6.

O gráfico apresenta os tempos registrados nas três execuções do experimento de falha controlada. Observa-se que os tempos permaneceram próximos, com média de 0,002523 segundo. Em todas as execuções, a exceção `FileNotFoundError` foi identificada e tratada, demonstrando o funcionamento do mecanismo de tratamento de erros implementado.

8.4.5 Relação com Sistemas Operacionais

O experimento C6 está relacionado ao gerenciamento de arquivos e às operações de entrada e saída.

Ao tentar abrir um arquivo inexistente, a aplicação solicita ao sistema operacional uma operação sobre o sistema de arquivos.

Como o recurso solicitado não está disponível, a operação falha e a aplicação recebe a indicação de erro, representada no Python pela exceção `FileNotFoundError`.

O experimento demonstra a importância de implementar mecanismos de tratamento de falhas para melhorar a robustez das aplicações.

8.5 Consolidação dos experimentos e análise estatística

Além da execução de C5 e C6, foi realizada a organização dos resultados disponíveis para a comparação das configurações experimentais.

O documento de consolidação foi criado no repositório:

`experimentos/consolidacao_c1_c6.md`

Foram reunidos os resultados dos experimentos C4, C5 e C6 disponíveis na etapa de consolidação.

A análise também considerou o número de repetições, a média dos tempos, o desvio-padrão amostral, os valores mínimos e máximos.

Tabela 3 – Estatísticas dos tempos de execução de C5 e C6

Configuração	Média (s)	Desvio-padrão (s)	Mínimo (s)	Máximo (s)
C5 – 100 MiB	0,138452	0,073089	0,095836	0,222847
C5 – 250 MiB	0,280650	0,098335	0,214362	0,393632
C5 – 500 MiB	0,554763	0,257336	0,371561	0,848971
C6 – Arquivo inexistente	0,002523	0,000357	0,002272	0,002932

Na C5, a carga de 500 MiB apresentou a maior dispersão absoluta dos tempos de execução.

Na C6, todas as execuções produziram o resultado esperado de falha tratada.

Como foram realizadas apenas três repetições por cenário, as estatísticas devem ser interpretadas como resultados preliminares.

A consolidação também incorporou os dados da configuração C4, referente ao cache compartilhado, permitindo relacionar os conceitos de localidade de acesso, políticas de substituição e concorrência.

Entretanto, os tempos de C4, C5 e C6 não devem ser comparados diretamente, pois representam operações diferentes e foram coletados em condições experimentais distintas.

8.6 Dificuldades e limitações

A principal preocupação durante o experimento C5 foi evitar que o aumento da utilização de memória comprometesse a estabilidade do computador.

Por esse motivo, as cargas foram limitadas a 500 MiB, com crescimento gradual e controlado.

Essa decisão permitiu realizar as medições com segurança, mas não provocou pressão suficiente para observar utilização de swap.

Outra limitação é que os experimentos foram realizados por meio de scripts sintéticos independentes.

Na C5, o aumento da memória foi simulado por alocações controladas, sem aumentar diretamente o corpus, o contexto ou o lote de requisições do modelo de IA.

Na C6, a falha foi simulada pela ausência de um arquivo, e não durante uma requisição real ao Ollama ou ao servidor integrado.

Portanto, os resultados caracterizam o comportamento dos cenários simulados, não o desempenho completo do sistema de IA em produção.

8.7 Conclusão

Os experimentos C5 e C6 permitiram relacionar conceitos fundamentais de Sistemas Operacionais a situações práticas de utilização de memória e tratamento de falhas.

Na C5, verificou-se que o aumento da carga provocou crescimento do tempo de execução e do consumo de memória residente. Apesar disso, não houve utilização de swap nem faltas maiores de página nas condições avaliadas.

Na C6, todas as tentativas de acesso ao arquivo inexistente foram identificadas e tratadas corretamente, demonstrando a importância do tratamento de exceções para a robustez das aplicações.

A consolidação das métricas e dos gráficos contribuiu para organizar as evidências experimentais e facilitar a interpretação dos resultados.

Como continuidade, recomenda-se executar os mesmos cenários diretamente no sistema integrado ao Ollama, aumentando gradualmente o contexto ou a quantidade de requisições, sempre respeitando os limites de memória disponíveis.

Dessa forma, a atividade contribuiu para compreender o gerenciamento de memória, a memória virtual, as operações de entrada e saída e os mecanismos de tratamento de erros em sistemas computacionais.

9 - Referências:

GOOGLE. *Gemma 4n*. Google AI for Developers. Disponível em: https://ai.google.dev/gemma/docs/gemma-4n?hl=pt-br. Acesso em: 15 set. 2026.

GOOGLE. *gemma-4-E2B-it-qat-q4_0-gguf*. Hugging Face. Disponível em: https://huggingface.co/google/gemma-4-E2B-it-qat-q4_0-gguf. Acesso em: 15 set. 2026.

OLLAMA. *gemma4n:e2b*. Ollama Library. Disponível em: https://ollama.com/library/gemma4n:e2b. Acesso em: 15 set. 2026.

- **Declaração de uso de IA generativa**

IA generativa:	Finalidade:	Prompt utilizado:	Saída:
Claude	Planejamento geral da atividade a partir do enunciado	"Analise TODO esse documento, e a partir da análise, quero sua ajuda com um roadmap para dividir as atividades entre os membros de forma coerente"	Sugeriu a divisão por seções do enunciado (ambiente, camada de aplicação, modelo, processos/threads, syscalls, experimentos) e recomendou o tópicos para cada membro do time, além disso, retornou um roteiro dia a dia (instalação, observação de processos/threads/syscalls, experimentos comparativos, relatório, vídeo)
Copilot (Github)	Organização da estrutura do repositório no GitHub	“A partir do Roadmap e divisão das tarefas anteriores, me ajude na criação de toda a estrutura de pastas no GitHub, de forma que facilite o trabalho dos membros”	Criou toda a estrutura do repositório, indicou em quais arquivos cada um iria ficar, a partir da pasta contexto/, onde cada trecho do texto deveria entrar, e os comandos git add/commit/push para subir o conteúdo para a branch main do nosso repositório.
Claude	Verificação de informação crítica	“Eu preciso da ficha técnica do modelo 4n-e2b escolhido pela equipe, use esse link: [link do gemma-4-E2B-it-qat-q4_0-gguf] e do modelo gemma-3-E2B e suas principais diferenças para determinar qual o mais indicado para a trilha escolhida pela equipe”	A IA alertou sobre a divergência de família de modelo e refez a ficha técnica com os dados corretos. Apontando o 4-E2B como o mais indicado para a atividade.
Claude	Aprofundamento em um critério específico	“Descreva o mais afundo o critério formato exigido na ficha técnica”	Explicação da cadeia de derivação do modelo (checkpoint base → instruct → QAT → conversão GGUF) e da relevância de cada etapa para os conceitos de Sistemas Operacionais (armazenamento, I/O, chamadas de sistema no carregamento)
Copilot	Apoio na escrita/organização do relatório	"Revise esse parágrafo do relatório em pdf do meu grupo, verifique erros de pontuação e duplicidade de palavras"	revisou o arquivo conforme solicitado e corrigiu erros de gramática e coesão
(IA QUE VCS USARAM)			